Отчёт по лабораторной работе № 2

по курсу «Операционные системы»

Вариант 19

Студент группы М8О-203БВ-25: Прудников Я. А.

Тема

Управление процессами в ОС Linux. Обмен данными между процессами с помощью каналов.

Цель

Получить практические навыки создания дочерних процессов, передачи данных через каналы и перенаправления стандартных потоков ввода и вывода.

Задание

Написать программу на языке Си, в которой родительский процесс создаёт два дочерних процесса. Родительская и дочерняя программы должны быть отдельными исполняемыми файлами.

Первыми двумя строками пользователь вводит имена файлов для записи результатов первого и второго дочерних процессов. После этого пользователь вводит строки произвольной длины. С вероятностью 80% родитель отправляет строку в `pipe1`, иначе в `pipe2`. Дочерние процессы удаляют гласные из полученных строк и записывают результат через стандартный вывод в соответствующие файлы. Необходимо предусмотреть обработку системных ошибок.

Идея, метод, алгоритм

Используем две программы: `parent` и `child`. Родитель запускает два экземпляра `child`. У каждого экземпляра свой входной канал и свой выходной файл, заданный пользователем.

1. Считываем имена двух файлов и удаляем из них символ перевода строки.
2. Создаём два неименованных канала.
3. Создаём два дочерних процесса. Их стандартный ввод перенаправляем на чтение из его канала, а стандартный вывод на запись в его файл.
4. Запускаем программу `child` в каждом дочернем процессе после `fork`.
5. Считываем очередную строку в родительском процессе. Получаем число `rand() % 100`. Если оно меньше 80, отправляем строку в первый канал, иначе во второй.
6. Дочерний процесс читает строку, удаляет из неё гласные и выводит результат.
7. После окончания ввода закрываем оба конца каналов для записи. Дочерние

процессы получают конец потока и завершаются

8. Дожидаемся завершения обоих процессов, проверяем их коды выхода и освобождаем ресурсы.

Реализация

Организация обмена данными

Стандартный ввод родителя связан с пользовательским вводом. В дочерних процессах `stdin` связан с каналом, а `stdout` с выходным файлом. Поэтому программа `child` работает с обычными потоками и не выбирает канал или файл самостоятельно.

Каналы создаём через `pipe2` с флагом `O_CLOEXEC`. Это сделано для того, чтобы каналы, унаследованные от родителя файловые дескрипторы при вызове `exec` автоматически закрывались. Процессы создаём через `fork`, потоки перенаправляем через `dup2`, а программу `child` запускаем через `execvp`. После замены программы исходные дескрипторы каналов закрываются благодаря `O_CLOEXEC`. Перенаправленные стандартные потоки остаются открытыми.

Файлы открываем с флагами `O_WRONLY | O_CREAT | O_TRUNC`. Если файл уже существует, его прежнее содержимое удаляется. Для результатов используем два разных имени файла. Для ожидания используем `waitpid`.

Обработка строк

В функции `read_line` сначала выделяем буфер на 256 байт. Если места не хватает, увеличиваем его в два раза через `realloc`. Это позволяет читать строки без заранее заданного ограничения длины, пока хватает памяти. Символ перевода строки сохраняем. Последнюю строку обрабатываем и в случае, если она заканчивается сразу концом потока

В функции `remove_vowels` используем два индекса. Первым просматриваем исходную строку, вторым отмечаем место для следующего сохраняемого символа. Если символ не является гласной, записываем его на это место. В конце ставим нулевой символ.

Распределение и запись строк, завершение работы

Значения от генератора от 0 до 79 направляют строку в первый канал, остальные значения во второй. Такое правило задаёт указанное в условии правило распределения

Функция `os_pipe_write` передаёт строку полностью. Если `write` записал только часть данных, продолжаем запись оставшейся части. При завершении вызываем `cleanup`: функция закрывает все записи в каналы, ожидает процессы, освобождает строки с именами файлов, структуры каналов и процессов

Проверяем результаты выделения памяти, создания каналов и процессов, открытия файлов, перенаправления потоков, записи и ожидания. При обнаруже-

нии ошибки возвращаем ненулевой код завершения. Для ряда системных ошибок выводим сообщение через `perror`.

Файловая структура программы

- `src/parent.c`: чтение пользовательского ввода, выбор канала и завершение работы
- `src/child.c`: чтение строк, удаление гласных и вывод результата.
- `src/line_utils.c`: утилита для чтения и работы со строками
- `src/platform/os_linux.c`: обёртка над os_api для ОС Linux.
- `src/include/os_api.h` и `src/include/line_utils.h`: объявления функций и типов

Код

Алгоритм выбора канала в родительской программе (фрагмент `parent.c`)

```
int choice = rand() % 100;
if (choice < 80) {
    if (os_pipe_write(pipe1, line, length) < 0) {
        cleanup(pipe1, pipe2, process_1, process_2,
                file1, file2);
        free(line);
        return 1;
    }
    free(line);
} else {
    if (os_pipe_write(pipe2, line, length) < 0) {
        cleanup(pipe1, pipe2, process_1, process_2,
                file1, file2);
        free(line);
        return 1;
    }
    free(line);
}
```

Перенаправление потоков в дочернем процессе (фрагмент `os_linux.c`)

```
if (close(input->fd[1]) == -1) {
    perror("close");
    _exit(1);
}
if (dup2(input->fd[0], STDIN_FILENO) == -1) {
    perror("dup2");
    _exit(1);
}
if (input->fd[0] != STDIN_FILENO) {
    if (close(input->fd[0]) == -1) {
        perror("close");
        _exit(1);
```

```
    }
}
int file_fd = open(output_file, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (file_fd == -1) {
    perror("open");
    _exit(1);
}
if (dup2(file_fd, STDOUT_FILENO) == -1) {
    perror("dup2");
    _exit(1);
}
if (file_fd != STDOUT_FILENO) {
    if (close(file_fd) == -1) {
        perror("close");
        _exit(1);
    }
}
char *args[] = {(char *)executable, NULL};
execvp(executable, args);
perror("execvp");
_exit(127);
```

Дочерняя программа (`child.c`)

```
#include "include/line_utils.h"
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>

int main() {
    while (true) {
        char *line = NULL;
        size_t length = 0;
        int result = read_line(stdin, &line, &length);
        if (result == 0) {
            free(line);
            break;
        }
        if (result == -1) {
            free(line);
            return 1;
        }
        remove_vowels(line);
        if (fputs(line, stdout) == EOF) {
            free(line);
            return 1;
        }
        free(line);
    }
    return 0;
}
```

Удаление гласных (фрагмент `line_utils.c`)

```c
void remove_vowels(char *line) {
    if (line == NULL) {
        return;
    }
    size_t j = 0;
    for (size_t i = 0; line[i] != '\0'; i++) {
        char ch = line[i];
        int is_vowel = (ch == 'a' || ch == 'e' ||
            ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' ||
            ch == 'O' || ch == 'U');
        if (!is_vowel) {
            line[j] = ch;
            j++;
        }
    }
    line[j] = '\0';
}
```

Тестирование

Входные данные	Полученный результат

`Hello World`
`Operating Systems`
`AEIOU aeiou`
	`Hll Wrld`
	`prtng Sstms`
	Один пробел, затем перевод строки

Пустая строка с переводом строки	Пустая строка с переводом строки
3000 букв `a`, затем `BC` и перевод строки	`BC` и перевод строки
`queue` без перевода строки в конце	`q` без перевода строки в конце
Пустой входной поток	Пустой вывод
Два имени файлов, затем конец ввода	Код `0`; оба файла пустые
Строка из 3000 цифр `0`	Одна строка длиной 3000 символов; код завершения `0`
Первое имя файла пустое, второе корректное	Код завершения `1`; файлы не созданы
Первый файл в несуществующей папке, второй путь корректный	Ошибка открытия файла; код завершения `1`; второй файл пустой

Вывод

В ходе работы научился связывать отдельные процессы через неименованные каналы и перенаправлять их стандартные потоки. Разобрался, что после `fork` нужно настроить файловые дескрипторы, а `execvp` запускает другую программу в уже созданном процессе. На примере завершения ввода понял, почему сначала

нужно закрывать запись в каналы и только потом ждать дочерние процессы: иначе они могут продолжать ожидать данные.

Также закрепил работу с динамической памятью при чтении длинных строк и обработку строки без дополнительного буфера.